

Documentación Técnica y de Implementación: Captura la Bandera Multijugador (C++)

Eduardo Andrés Tobar Hernández - 23001175

Proyecto: Captura la Bandera (CTF) Multijugador en Tiempo Real

Lenguaje y Tecnologías: C++17, Sockets Nativos (POSIX / Winsock), Raylib (Interfaz Gráfica 2D), Catch2 (Pruebas Unitarias e Integración), nlohmann/json (Serialización JSON), CMake y vcpkg.

Arquitectura: Modelo Cliente-Servidor Autoritativo con Arquitectura Híbrida TCP/UDP.

1. Resumen Ejecutivo del Sistema

El proyecto consiste en una aplicación multijugador en tiempo real basada en el clásico juego "Captura la Bandera" (CTF). El software se ha construido íntegramente en C++17 cumpliendo con los estándares de interoperabilidad de la clase (Protocolo CTF v1.2.0 / v=1). El sistema soporta hasta 100 conexiones simultáneas por TCP y proporciona descubrimiento automático de servidores en red local mediante UDP Broadcast.

Toda la lógica de física, movimiento, validación de reglas, administración de bandera y condiciones de victoria reside de forma estricta en el **servidor autoritativo**. Los clientes operan enviando intenciones de movimiento e interacción, recibiendo ráfagas de estado a 20 Hz para renderizar la partida en un entorno gráfico interactivo desarrollado con Raylib.

2. Arquitectura de Red y Protocolo de Comunicación

2.1 Esquema Híbrido de Transporte

- **TCP (Puerto de Juego):** Utilizado para todo el flujo principal de la partida (unión de jugadores, envío de intenciones de movimiento, captura y robo de bandera, difusiones de estado y finalización). Garantiza la entrega ordenada y confiable de datos.
- **UDP (Puerto 8888 de Descubrimiento):** Reservado exclusivamente para la localización automática de servidores en la red local mediante doble difusión (Broadcast limitado 255.255.255.255 y Broadcast de subred) y solicitudes Unicast de respaldo.

2.2 Framing de Mensajes y Coalescencia

Debido a que TCP entrega un flujo continuo de bytes, el sistema implementa una capa de delimitación basada en un acumulador de bytes (LineBuffer) que separa los mensajes por el carácter de salto de línea (\n), con tolerancia a terminaciones tipo Windows (\r\n). Los mensajes que superen los 64 KB son rechazados de inmediato.

Para evitar saturar a clientes lentos con acumulación de fotogramas desfasados, el servidor aplica **coalescencia de estados**: si un mensaje de tipo state está pendiente de envío hacia un cliente y llega una nueva actualización de física, el estado antiguo se reemplaza en cola por el más reciente. Mensajes de control como lobby, countdown, start y game_over nunca se coalescen ni se descartan.

3. Estructura y Módulos del Código Fuente

Módulo / Directorio	Componentes Principales	Descripción y Responsabilidad
src/common/	constants.hpp, json.hpp, framing.hpp, messages.hpp, gui_helpers.hpp	Definiciones del protocolo v1.2.0, utilidades JSON, delimitación TCP (LineBuffer), catálogo completo de mensajes con validación estricta de campos y paleta gráfica compartida.
src/common/net/	tcp_socket.hpp, udp_socket.hpp, poller.hpp, platform.hpp	Abstracción de red multiplataforma sin dependencias externas (POSIX Sockets en macOS/Linux, Winsock2 en Windows) con soporte de sockets no bloqueantes y multiplexación via Poller.
src/server/	server.hpp, game.hpp, server_view.hpp	Servidor de juego con bucle de eventos a 20 Hz, máquina de fases, validación de física CTF, administración de desconexiones y vista gráfica de observador.
src/client/	client.hpp, input.hpp, game_view.hpp	Aplicación cliente completa con interfaz Raylib, búsqueda de servidores, salas de espera, renderizado 2D de partida, teclado e inferencia de salidas.
tests/	test_framing.cpp, test_messages.cpp, test_domain.cpp, test_integration.cpp	Suite de pruebas automatizadas con Catch2 que cubren framing, catálogo de mensajes, física del dominio y simulaciones multijugador con clientes automáticos (bots).

4. Cronología y Fases de Desarrollo

Fase 1: Fundamentos y Capa de Protocolo

Se comenzó estableciendo la estructura del proyecto en CMake con gestión de dependencias mediante vcpkg. Se definieron las constantes globales del estándar (mapa de 1000x1000, radio central de 300, velocidad de 200 u/s, radio de victoria de 315, puerto UDP 8888, límite de 100 jugadores) de forma inmutable. Posteriormente, se construyeron los módulos de parsing JSON seguro y la delimitación de líneas TCP en el objeto LineBuffer.

Se implementó el catálogo completo de mensajes (discover, server_info, join, input, interact, welcome, lobby, countdown, start, state, game_over, error) mediante tipos de datos fuertemente tipados en C++ y validación estructurada por campo.

Scaffold the project structure for `ctf-multiplayer-cpp`:

1. Create root `CMakeLists.txt` with:
 - C++17 project, single executable named `ctf`
 - Subdirectories: `src/common/`, `src/server/`, `src/client/`, `tests/`
 - `find_package(nlohmann_json CONFIG REQUIRED)`, `find_package(raylib CONFIG REQUIRED)`, `find_package(Catch2 CONFIG REQUIRED)`
 - Tests with Catch2: `add_test` + `enable_testing`, target `ctf_tests`
2. Create `CMakePresets.json` with:
 - `default` (macOS/Linux, Clang or GCC, vcpkg toolchain)
 - `release` (inherits from default, Release)
 - `windows-msvc` (Windows, MSVC, vcpkg toolchain)
 - Use appropriate `VCPKG_TARGET_TRIPLET`.
3. Create `vcpkg.json` (manifest mode) with dependencies:
 - `nlohmann-json`
 - `raylib`
 - `catch2`
4. Create `vcpkg-configuration.json` with a fixed baseline.
5. Create empty directories: `src/common/`, `src/server/`, `src/client/`, `tests/`, `docs/`
6. Create a minimal `src/main.cpp` that parses `--server` / `--client` from `argc/argv` and prints "server mode" or "client mode".
7. Verify that `cmake --preset=default && cmake --build --preset=release` compiles without errors (the `find_package` calls will likely fail until vcpkg is installed; if so, at least verify the CMake structure is syntactically valid with `cmake -S . -B build_test` without a toolchain).

Create `src/common/constants.hpp` with `#pragma once` and a `ctf::constants` namespace (or `namespace ctf { inline namespace constants { ... }}`). Define all SPEC v1.2.0 constants as `constexpr`:

```
inline constexpr int map_size = 1000;
inline constexpr int circle_radius = 300;
inline constexpr int player_radius = 15;
inline constexpr int interact_radius = 40;
inline constexpr int speed = 200;           // units/second
inline constexpr int tick_rate = 20;        // Hz
inline constexpr int countdown_seconds = 5;
inline constexpr int min_players = 2;
inline constexpr int post_game_seconds = 5;
inline constexpr int circle_center_x = 500;
inline constexpr int circle_center_y = 500;
inline constexpr int spawn_radius_min = 350;
inline constexpr int spawn_radius_max = 450;
inline constexpr int victory_distance = 315; // circle_radius + player_radius
inline constexpr int discovery_port = 8888;  // UDP
inline constexpr int max_players = 100;
inline constexpr int name_max_length = 20;
inline constexpr int message_max_size = 64 * 1024; // 64 KB
inline constexpr int default_tcp_port = 8889;
```

All values must match the "Agreed Constants and Limits" table in SPEC v1.2.0 exactly. Header-only file, no .cpp.

Using `inline constexpr` (C++17) is helpful to avoid ODR issues when included from multiple TUs.

Create `src/common/json.hpp` and `src/common/json.cpp`:

- Namespace `ctf::json` (or inside `ctf::common`).
- Header: include `<nlohmann/json.hpp>`.
- Function `auto parse_line(std::string_view line) -> std::optional<nlohmann::json>`:
 - Uses `nlohmann::json::parse()` to parse the string.
 - Catches parse exceptions and returns `std::nullopt` on invalid JSON.
 - Tolerant parsing: accepts JSON with surrounding whitespace.
- Function `auto dump_compact(const nlohmann::json& j) -> std::string`:
 - Returns `j.dump(-1, ' ', false, nlohmann::json::error_handler_t::ignore)` — no spaces, no newlines.
 - No unnecessary escape characters.
- Don't add extra functionality; this is a minimal wrapper.

Implementation in a separate .cpp because `nlohmann::json` is slow to compile; the wrapper isolates it.

Create `src/common/framing.hpp` and `src/common/framing.cpp`:

Class `LineBuffer`:

- Stores a `std::string` as internal accumulation buffer.
- Constructor: receives `max_size` (default `message_max_size = 65536`).
- `void append(const char* data, size_t len)`: adds bytes to the buffer. If the total buffer exceeds `max_size`, throws `message_too_large_error` (custom exception with an `error_code()` method returning `"MESSAGE_TOO_LARGE"`).
- `auto extract() -> std::optional<std::string>`: looks for the first `\n` in the buffer. If found:
 - Extracts the substring before `\n`.
 - If the last character before `\n` is `\r`, discards it (`\r\n` tolerance).
 - Removes everything up to and including `\n` from the buffer.
 - Returns the extracted message.
 - If no `\n`, returns `std::nullopt`.
- `void reset()`: clears the buffer.
- `bool empty() const`: whether the buffer is empty.

Free function `encode`:

- `auto encode(const nlohmann::json& msg) -> std::string`: Returns `dump_compact(msg) + "\n"`, checks that it doesn't exceed `message_max_size` (if it does, throws an exception).

Include necessary headers in `.cpp`. Reference `constants.hpp` and `json.hpp`.

For example:

- Input: `{"a":1}\n{"b":2}\n` -> two messages.
- Input: `{"a":}` -> `nullopt` (missing `\n`).
- Input: `{"a":1}\r\n` -> message `{"a":1}` (discards `\r`).

Create `src/common/messages.hpp` and `src/common/messages.cpp`:

Data structs (in `namespace ctf`):

```
struct Discover { int v; };
struct ServerInfo { int v; std::string name; int tcp_port; std::string state; int players; };
struct Join { int v; std::string name; };
struct Input { int dir_x; int dir_y; };
struct Interact {}; // no fields
struct Welcome { std::string player_id; Config config; };
struct Config { int map_size; int circle_radius; int player_radius; int interact_radius; int speed; int tick_rate; };
struct Lobby { std::vector<PlayerInfo> players; };
struct PlayerInfo { std::string id; std::string name; };
struct Countdown { int seconds; };
struct Start {}; // no fields
struct State { Flag flag; std::vector<Player> players; };
struct Flag { std::optional<std::string> owner; double x; double y; };
struct Player { std::string id; double x; double y; };
struct GameOver { std::string winner; };
struct Error { std::string reason; };
```

Serialization and deserialization:

Functions in `namespace ctf::msg`:

- `auto from_json(const nlohmann::json& j) -> std::variant<Discover, ServerInfo, Join, Input, Interact, Welcome, Lobby, Countdown, Start, State, GameOver, Error>`:
 - Reads `j["type"]` as a string and dispatches to the specific parser.
 - Each parser validates:
 - Required fields are present (if missing: `MISSING_FIELD`).
 - Correct types (if type is wrong or out of range: `INVALID_FIELD`).

Fase 2: Red Multiplataforma y Descubrimiento UDP

Se desarrolló la abstracción de sockets nativos soportando POSIX y Winsock sin requerir librerías de conexión de terceros. Se habilitaron las opciones de socket `SO_REUSEADDR` y `SO_REUSEPORT` para posibilitar ejecuciones y pruebas múltiples en la misma máquina local. El servicio de descubrimiento `DiscoveryServer` responde a solicitudes UDP en el puerto 8888 informando el puerto TCP activo del servidor, estado y número de jugadores. El `DiscoveryClient` realiza escaneos por broadcast dual y soporta conexiones manuales directas mediante dirección IP y puerto.

Build the entire networking layer in `src/common/net/` and `src/common/udp_discovery.hpp/.cpp`:

Part A — Socket abstraction (`src/common/net/`)

- `src/common/net/platform.hpp` — platform detection:
 - `#ifdef _WIN32` → include `winsock2.h`, `ws2tcpip.h`; `#pragma comment(lib, "ws2_32")`.
 - `#else` → POSIX headers.
 - `auto init_net()` → `bool` (calls `WSAStartup` on Windows, no-op on POSIX).
 - `auto cleanup_net()` → `void` (calls `WSACleanup` on Windows).
- `src/common/net/socket.hpp` — type aliases:
 - `using socket_t = int`; on POSIX, `using socket_t = SOCKET`; on Windows.
 - `constexpr socket_t invalid_socket = -1`; (POSIX) or `INVALID_SOCKET` (Windows).
 - `auto close_socket(socket_t)` → `void`;
 - `auto last_error()` → `std::string`;
- `src/common/net/tcp_socket.hpp/.cpp` — `class TcpSocket` (non-copyable, movable, RAI):
 - `static auto listen(int port)` → `TcpSocket` — creates TCP socket, `bind`, `listen`, `SO_REUSEADDR`, non-blocking.
 - `auto accept()` → `std::optional<TcpSocket>` — accepts non-blocking.
 - `auto recv(char* buf, size_t len)` → `ssize_t` — returns -1 on `EAGAIN/EWOULDBLOCK`, 0 on closed.
 - `auto send(const char* data, size_t len)` → `ssize_t`.
 - `auto native_handle()` → `socket_t`, `close()`.
- `src/common/net/udp_socket.hpp/.cpp` — `class UdpSocket` (non-copyable, movable):
 - `static auto bind(int port)` → `UdpSocket` — UDP socket, `SO_REUSEADDR + SO_REUSEPORT`, `bind` to `INADDR_ANY`.
 - `static auto make_broadcast()` → `UdpSocket` — UDP socket with `SO_BROADCAST`.
 - `auto send_to(const std::string& data, const sockaddr_in& addr)` → `bool`.
 - `auto recv_from(char* buf, size_t len, sockaddr_in& sender)` → `ssize_t`.
- `src/common/net/poller.hpp/.cpp` — `class Poller`:

Part B — UDP discovery server (`src/common/udp_discovery.hpp/.cpp`)

Class `DiscoveryServer`:

- Takes a reference to `UdpSocket` and a `ServerStateProvider` interface.
- `void start(int discovery_port)`.
- `void tick()`: non-blocking `recv_from` loop:
 - Checks size $\leq$ `message_max_size` (silently discards).
 - Parses JSON; if `{"type": "discover", "v": 1}` $\rightarrow$ builds `server_info` and sends `send_to()` back to sender.
 - `v != 1` or invalid $\rightarrow$ silently discards.

Interface `ServerStateProvider`:

- `virtual auto get_server_name() const -> std::string = 0;`
- `virtual auto get_tcp_port() const -> int = 0;`
- `virtual auto get_state() const -> std::string = 0; // "lobby" or "playing"`
- `virtual auto get_player_count() const -> int = 0;`

Part C — UDP discovery client (same file, `src/common/udp_discovery.hpp/.cpp`)

Class `DiscoveryClient`:

- `void discover(int port, int timeout_ms = 2000)`:
 - Create UDP socket with `SO_BROADCAST`.
 - Build `{"type": "discover", "v": 1}`, encode.
 - Send to both: `255.255.255.255:port` and subnet broadcast (compute via `getifaddrs()` on POSIX, `GetAdaptersInfo()` on Windows).
 - Poll for responses up to `timeout_ms`; parse `server_info` with `v==1`, store in `std::vector<ServerEntry>`.
- `struct ServerEntry { std::string ip; std::string name; int tcp_port; std::string state; int players; };`
- `auto servers() const -> const std::vector<ServerEntry>&`.
- `void clear()`.

Manual mode: `auto discover_unicast(const std::string& ip, int port, int timeout_ms = 1000) -> std::optional<ServerEntry>` sends unicast `discover`, waits for one response.

Direct fallback: Function to parse `IP:port` format and skip discovery entirely (go straight to TCP).

Fase 3: Servidor Autoritativo y Dominio del Juego

Se construyó el motor de juego del servidor operando en un bucle no bloqueante administrado por Poller. Se implementó la máquina de fases del juego:

- **LOBBY:** Recibe conexiones, valida nombres (1 a 20 caracteres UTF-8), asigna IDs únicos y envía la configuración en el mensaje welcome. Al alcanzar al menos 2 jugadores, se inicia la cuenta regresiva.
- **COUNTDOWN:** Emite mensajes de cuenta regresiva de 5 a 1 segundo. Si la cantidad de jugadores cae por debajo del mínimo, el conteo se cancela y se regresa al estado de espera.
- **PLAYING:** Ejecuta la simulación física a 20 Hz. Procesa movimiento con normalización en diagonales (división por $\sqrt{2}$), limita posiciones dentro del mapa (15 a 985 unidades), gestiona capturas/robos de bandera y valida la condición de victoria.
- **POST_GAME:** Muestra al ganador durante 5 segundos y reinicia automáticamente la partida hacia la fase LOBBY sin desconectar a los participantes.

Implement the core server: TCP listener, session management, phase machine, join/lobby flow, and countdown logic.

Part A — TCP server skeleton (`src/server/server.hpp/.cpp`, `src/server/main.cpp`)

Class `Session` (per connection):

- `socket_t fd;`
- `LineBuffer recv_buf;`
- `std::queue<std::string> send_queue;` (pending messages)
- `std::optional<std::string> coalesced_state;` — pending state for coalescing.
- `bool wants_write = false;` — registered for writing.
- `void queue_message(const std::string& msg);` — enqueues or coalesces.
- `auto try_send() -> bool;` — drains `send_queue`; returns false on error.

Class `Server`:

- `TcpSocket listener;` (bind + listen on `default_tcp_port`).
- `Poller poller;`
- `std::map<socket_t, std::unique_ptr<Session>> sessions;`
- Current phase (enum class `Phase { Lobby, Countdown, Playing, PostGame }`).
- `void run();` — main loop:
 - `poller.poll(50)` (20 Hz $\approx$ 50 ms per tick).
 - Accept new connections $\rightarrow$ create `Session` $\rightarrow$ `poller.add_fd`.
 - For each readable session: `append` + extract messages in loop; each $\rightarrow$ `dispatch_message(session, msg)`.
 - For each writable session: `try_send()`; if fails $\rightarrow$ `disconnect(socket)`.
 - If phase == Playing: `game_tick()` (will be built later).
 - `cleanup_disconnected()`.

src/server/main.cpp:

- `init_net()`, parse `--server [port]`, create `Server`, `run()`, `cleanup_net()`.

Part B — Phase machine and join handling

LOBBY phase:

- On `join`:
 - Validate `v == 1` → `VERSION_MISMATCH` + close.
 - Validate `name` (trim 1-20, no control chars) → `NAME_INVALID` (no close).
 - Second join per session → `INVALID_PHASE` (no close).
 - `sessions.size() >= max_players` → `LOBBY_FULL` + close.
 - Assign unique `player_id`.
 - Send `welcome{player_id, config}` to the player, then `lobby`.
 - Broadcast `lobby{players[]}` to all in lobby.
 - If `sessions.size() >= min_players` (2) → start countdown.
- Non-`join` messages → `INVALID_PHASE`.

COUNTDOWN phase: `join` → `GAME_STARTED` + close. `input/interact` → `INVALID_PHASE`.

PLAYING phase: `join` → `GAME_STARTED` + close. `input/interact` → process game.

`broadcast_lobby()`: sends updated `lobby{players[]}` to all lobby sessions.

Part C — Countdown timer

- When `sessions.size() >= 2` in LOBBY → transition to COUNTDOWN, `countdown_remaining = 5`, send `countdown{seconds:5}`.
- Each poll loop iteration: accumulate elapsed time. Every 1 second:
 - `countdown_remaining--`
 - If `> 0`: broadcast `countdown{seconds}`.
 - If `== 0`: broadcast `start`, transition to PLAYING.
- If `sessions.size() < min_players` at any point during countdown:
 - Abort: broadcast `lobby`, `phase = Phase::Lobby`, reset timer.

Fase 4: Cliente Gráfico Interactivo

Se diseñó la aplicación cliente sobre Raylib, estructurada en pantallas independientes (Búsqueda de servidores, Ingreso de nombre, Sala de espera, Cuenta regresiva, Partida en curso, Pantalla de victoria y Pantalla de desconexión).

El cliente captura la entrada del teclado (WASD / Flechas) y envía intenciones de dirección (`dir.x` y `dir.y` en `{-1, 0, 1}`) únicamente cuando existe un cambio de dirección respecto al fotograma anterior, evitando saturar la red con mensajes idénticos. La tecla de interacción (E) permite solicitar la captura o robo de la bandera.

Fase 5: Verificación y Pruebas Automatizadas

Se incorporó un conjunto exhaustivo de pruebas unitarias e integración con la librería Catch2:

- **Pruebas de Framing:** Verificación de separación de mensajes múltiples en una lectura, recepción parcial de datos, tolerancia a `\r\n` y límites de 64 KB.
- **Pruebas de Validación:** Comprobación de errores como `VERSION_MISMATCH`, `INVALID_FIELD`, `NAME_INVALID` y `MISSING_FIELD`.
- **Pruebas del Dominio:** Verificación de que 100 apariciones de prueba ocurran en el anillo exterior (distancia entre 350 y 450), movimiento en ejes y diagonales, ajuste a bordes y condición de victoria estricta.
- **Pruebas de Integración con Bots:** Ejecución headless del servidor conectando múltiples clientes simulados (`BotClient`) que ejecutan ciclos completos de partida, cancelación de cuenta regresiva y desconexión de portadores.

Add an integration test in `tests/test_discovery.cpp` (Catch2).

Test case "UDP discovery local":

1. Initialize networking with `init_net()`.
2. Create a `DiscoveryServer` with a dummy provider (`name="TestServer", tcp_port=8889, state="lobby", players=0`).
3. Start on port 8888 (or a random port to avoid conflicts).
4. In a separate thread or sequentially, create `DiscoveryClient` and call `discover_unicast("127.0.0.1", port)`.
5. Verify:
 - At least one `ServerEntry` was received.
 - `entry.name == "TestServer"`.
 - `entry.tcp_port == 8889`.
 - `entry.state == "lobby"`.
6. Cleanup with `cleanup_net()`.

Test case "Silently discard invalid":

1. Send datagram `{"type": "discover", "v": 2}` → should not respond.
2. Send garbage text → should not respond.

Add the test to `CMakeLists.txt` under the `ctf_tests` target.

Commit: `test(net): add local UDP discovery integration test`

Fase 6: Experiencia Visual y Pulido de Interfaz (UI/UX)

En la etapa final de desarrollo se realizaron mejoras estéticas profundas tanto en el observador del servidor como en el cliente:

- **Cuadrícula y Campo de Juego:** Renderizado de cuadrícula suave en el fondo, anillo central con gradiente de transparencia e iluminación visual ("BASE"), indicación textual de bandera y límites visibles del mapa.
- **Elementos de Jugadores y Animaciones:** Anillo exterior decorativo en avatares, brillo pulsante alrededor del portador de la bandera, indicador triangular de dirección de movimiento, banderas animadas con efecto de oscilación y etiquetas con fondo oscuro para legibilidad.
- **Paneles de Interfaz y Ayuda:** Incorporación de la dirección IP local del servidor en pantalla, partículas ambientales en salas de espera, botón interactivo para inicio manual de partida, panel informativo de controles y reglas del juego ("How to Play" activable con la tecla H) y transiciones suaves de atenuación entre pantallas.

5. Reglas de Juego y Matriz de Validaciones

Acción del Juego	Entrada del Cliente	Validación Autoritativa en Servidor	Resultado y Efecto
Aparición (Spawn)	Señal start recibida	Generación aleatoria de ángulo $\theta \in [0, 2\pi)$ y radio $R \in [350, 450]$.	Jugador aparece fuera del círculo central. No hay colisión entre cuerpos.
Desplazamiento	Mensaje input con dir.x y dir.y	Comprueba que los valores sean -1, 0 o 1. Normaliza vectores diagonales ($\div\sqrt{2}$) e integra con velocidad de 200 u/s.	Actualiza la coordenada oficial. Ajusta la posición si intenta salir del rango [15, 985].
Captura de Bandera	Mensaje interact	Comprueba que la bandera esté libre y la distancia del jugador a la bandera sea ≤ 40 unidades.	Asigna el ID del jugador como portador (flag.owner). La bandera acompaña al jugador.
Robo de Bandera	Mensaje interact	Comprueba que la bandera tenga portador y la distancia del solicitante al portador sea ≤ 40 unidades.	Transfiere la bandera inmediatamente al nuevo jugador. Sin tiempos de espera ni inmunidad.
Victoria	Movimiento continuo	Evalúa si el portador realizó la transición física de estar dentro del círculo (distancia ≤ 315) a estar fuera (distancia > 315).	Finaliza la partida, emite el mensaje game_over y declara al ganador.

6. Guía de Compilación y Ejecución

6.1 Requisitos Previos

- Compilador compatible con C++17 (GCC, Clang o MSVC).
- CMake (versión 3.20 o superior).
- Administrador de paquetes vcpkg configurado en modo manifiesto.

6.2 Comandos de Compilación

```
# Configuración del proyecto mediante preset por defecto  
cmake --preset=default
```

```
# Compilación del ejecutable en modo Release  
cmake --build --preset=release
```

6.3 Comandos de Ejecución

```
# Ejecución del Servidor con vista de observador (puerto predeterminado 8889)  
./build/ctf --server
```

```
# Ejecución de un Cliente para unirse a partidas  
./build/ctf --client
```

```
# Ejecución de la suite de pruebas automatizadas  
ctest --test-dir build --output-on-failure
```